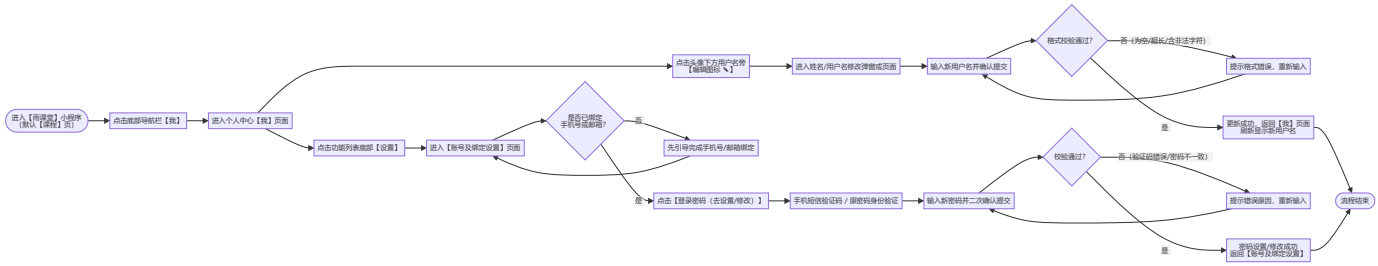
1030实验：微信小程序事件流与图书馆管理系统 UML 建模

第一部分：微信小程序（雨课堂）修改用户名与密码事件流

1.1 事件流 Mermaid 流程图




```mermaid
flowchart LR
    Start(["进入【雨课堂】小程序<br/>（默认【课程】页）"]) --> NavMe["点击底部导航栏【我】"]
    NavMe --> MePage["进入个人中心【我】页面"]

    %% 分支一：修改用户名
    MePage --> ClickEdit["点击头像下方用户名旁<br/>【编辑图标 ✎】"]
    ClickEdit --> EditModal["进入姓名/用户名修改弹窗或页面"]
    EditModal --> InputName["输入新用户名并确认提交"]
    InputName --> CheckName{"格式校验通过？"}
    CheckName -- "否（为空/超长/含非法字符）" --> ErrName["提示格式错误，重新输入"]
    ErrName --> InputName
    CheckName -- "是" --> UpdateName["更新成功，返回【我】页面<br/>刷新显示新用户名"]
    UpdateName --> EndNode(["流程结束"])

    %% 分支二：修改/设置登录密码
    MePage --> ClickSetting["点击功能列表底部【设置】"]
    ClickSetting --> AccountPage["进入【账号及绑定设置】页面"]
    AccountPage --> CheckBind{"是否已绑定<br/>手机号或邮箱？"}
    CheckBind -- "否" --> DoBind["先引导完成手机号/邮箱绑定"]
    DoBind --> AccountPage
    CheckBind -- "是" --> ClickPwd["点击【登录密码（去设置/修改）】"]
    ClickPwd --> VerifyID["手机短信验证码 / 原密码身份验证"]
    VerifyID --> InputPwd["输入新密码并二次确认提交"]
    InputPwd --> CheckPwd{"校验通过？"}
    CheckPwd -- "否（验证码错误/密码不一致）" --> ErrPwd["提示错误原因，重新输入"]
    ErrPwd --> InputPwd
    CheckPwd -- "是" --> UpdatePwd["密码设置/修改成功<br/>返回【账号及绑定设置】"]
    UpdatePwd --> EndNode
```

1.2 结构化事件流说明

流程分类	正常事件流（基本流）	备选/异常事件流（扩展流）
公共前置与入口	1. 用户登录「雨课堂」微信小程序，默认处于**「课程」**页。**2. 点击底部导航栏**「我」**进入个人中心页面。	若微信登录态失效，先触发微信授权登录再进入个人中心。
修改用户名	1. 在**「我」**页面点击头像下方用户名右侧的**「编辑图标（✎）」。**2. 弹出或跳转至姓名编辑界面，回显原用户名。**3. 输入新用户名并点击保存。**4. 系统校验通过后更新服务端数据，返回**「我」**页面并刷新显示新用户名。	**1a. 输入为空或超长**：提示格式错误，停留在编辑框重新输入。**1b. 取消修改**：用户点击取消/返回，丢弃更改并保持原用户名不变。
修改/设置登录密码	1. 在**「我」**页面点击列表底部的**「设置」。**2. 进入**「账号及绑定设置」**页面。**3. 点击**「登录密码」**一栏（去设置/去修改）。4. 系统确认已绑定手机号或邮箱后，通过短信验证码/原密码验证身份。5. 输入新密码并二次确认提交，校验通过后更新密码并返回设置页。	**2a. 未绑定手机号且未绑定邮箱**：因小程序要求先绑定手机号/邮箱才可设置密码登录网页端，系统先拦截并引导绑定手机号或邮箱。**2b. 验证码错误或两次密码不一致**：系统提示具体错误原因，要求重新输入。

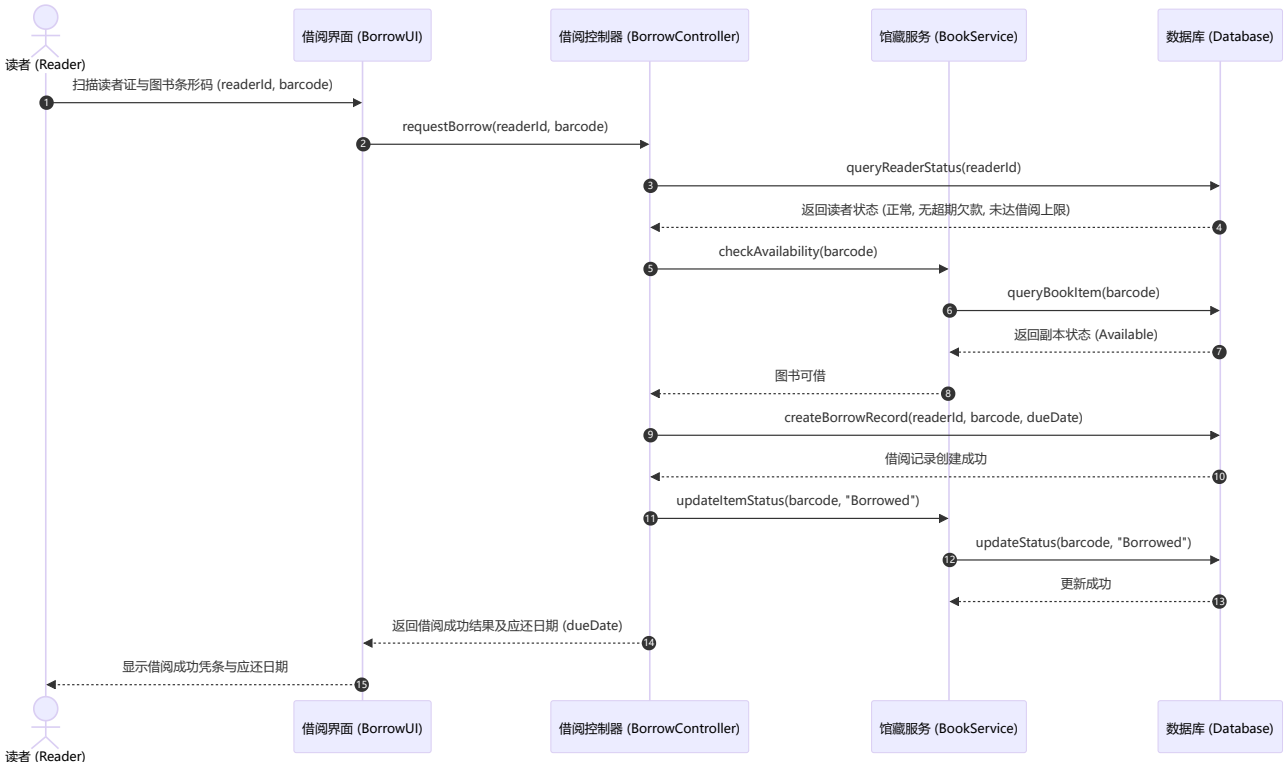
第二部分：图书馆管理系统（Library Management System）UML 图

> **说明**：以下严格按照实验要求列出图书馆管理系统的 **10 种 UML 图**（涵盖 `Sequence diagram`、`Use case diagram`、`Class diagram`、`Object diagram`、`Activity diagram`、`Legacy syntax`、`Component diagram`、`Deployment diagram`、`State diagram`、`Timing diagram`）。每一种图均同时提供 **Mermaid 代码**（可在 VS Code / Typora 中直接预览）与标准 **PlantUML 代码**（完全兼容 PlantUML 语法规范）。

1. Sequence diagram（时序图 / 顺序图）

描述**读者借阅图书**时，读者、借阅界面、借阅控制器、馆藏图书服务与数据库之间的消息交互顺序。

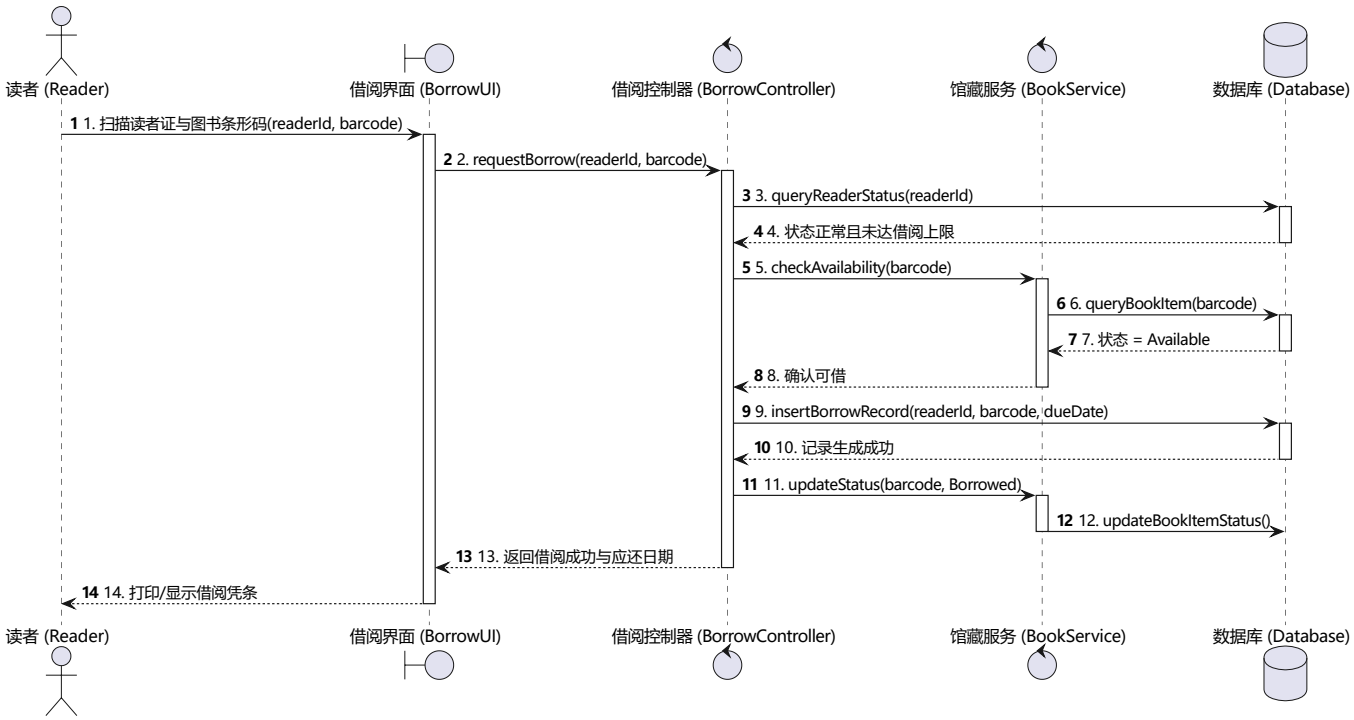
Mermaid 预览版




```
sequenceDiagram
    autonumber
    actor Reader as 读者 (Reader)
    participant UI as 借阅界面 (BorrowUI)
    participant Ctrl as 借阅控制器 (BorrowController)
    participant BookSvc as 馆藏服务 (BookService)
    participant DB as 数据库 (Database)

    Reader->>UI: 扫描读者证与图书条形码 (readerId, barcode)
    UI->>Ctrl: requestBorrow(readerId, barcode)
    Ctrl->>DB: queryReaderStatus(readerId)
    DB-->>Ctrl: 返回读者状态 (正常, 无超期欠款, 未达借阅上限)
    Ctrl->>BookSvc: checkAvailability(barcode)
    BookSvc->>DB: queryBookItem(barcode)
    DB-->>BookSvc: 返回副本状态 (Available)
    BookSvc-->>Ctrl: 图书可借
    Ctrl->>DB: createBorrowRecord(readerId, barcode, dueDate)
    DB-->>Ctrl: 借阅记录创建成功
    Ctrl->>BookSvc: updateItemStatus(barcode, "Borrowed")
    BookSvc->>DB: updateStatus(barcode, "Borrowed")
    DB-->>BookSvc: 更新成功
    Ctrl-->>UI: 返回借阅成功结果及应还日期 (dueDate)
    UI-->>Reader: 显示借阅成功凭条与应还日期
```

PlantUML 标准版


读者 (Reader)
借阅界面 (BorrowUI)
借阅控制器 (BorrowController)
馆藏服务 (BookService)
数据库 (Database)
1 1. 扫描读者证与图书条形码(readerId, barcode)
2 2. requestBorrow(readerId, barcode)
3 3. queryReaderStatus(readerId)
4 4. 状态正常且未达借阅上限
5 5. checkAvailability(barcode)
6 6. queryBookItem(barcode)
7 7. 状态 = Available
8 8. 确认可借
9 9. insertBorrowRecord(readerId, barcode, dueDate)
10 10. 记录生成成功
11 11. updateStatus(barcode, Borrowed)
12 12. updateBookItemStatus()
13 13. 返回借阅成功与应还日期
14 14. 打印/显示借阅凭条
读者 (Reader)
借阅界面 (BorrowUI)
借阅控制器 (BorrowController)
馆藏服务 (BookService)
数据库 (Database)

```
@startuml
autonumber
actor "读者 (Reader)" as Reader
boundary "借阅界面 (BorrowUI)" as UI
control "借阅控制器 (BorrowController)" as Ctrl
control "馆藏服务 (BookService)" as BookSvc
database "数据库 (Database)" as DB

Reader -> UI : 1. 扫描读者证与图书条形码(readerId, barcode)
activate UI
UI -> Ctrl : 2. requestBorrow(readerId, barcode)
activate Ctrl

Ctrl -> DB : 3. queryReaderStatus(readerId)
activate DB
DB --> Ctrl : 4. 状态正常且未达借阅上限
deactivate DB

Ctrl -> BookSvc : 5. checkAvailability(barcode)
activate BookSvc
BookSvc -> DB : 6. queryBookItem(barcode)
activate DB
DB --> BookSvc : 7. 状态 = Available
deactivate DB
BookSvc --> Ctrl : 8. 确认可借
deactivate BookSvc

Ctrl -> DB : 9. insertBorrowRecord(readerId, barcode, dueDate)
activate DB
DB --> Ctrl : 10. 记录生成成功
deactivate DB

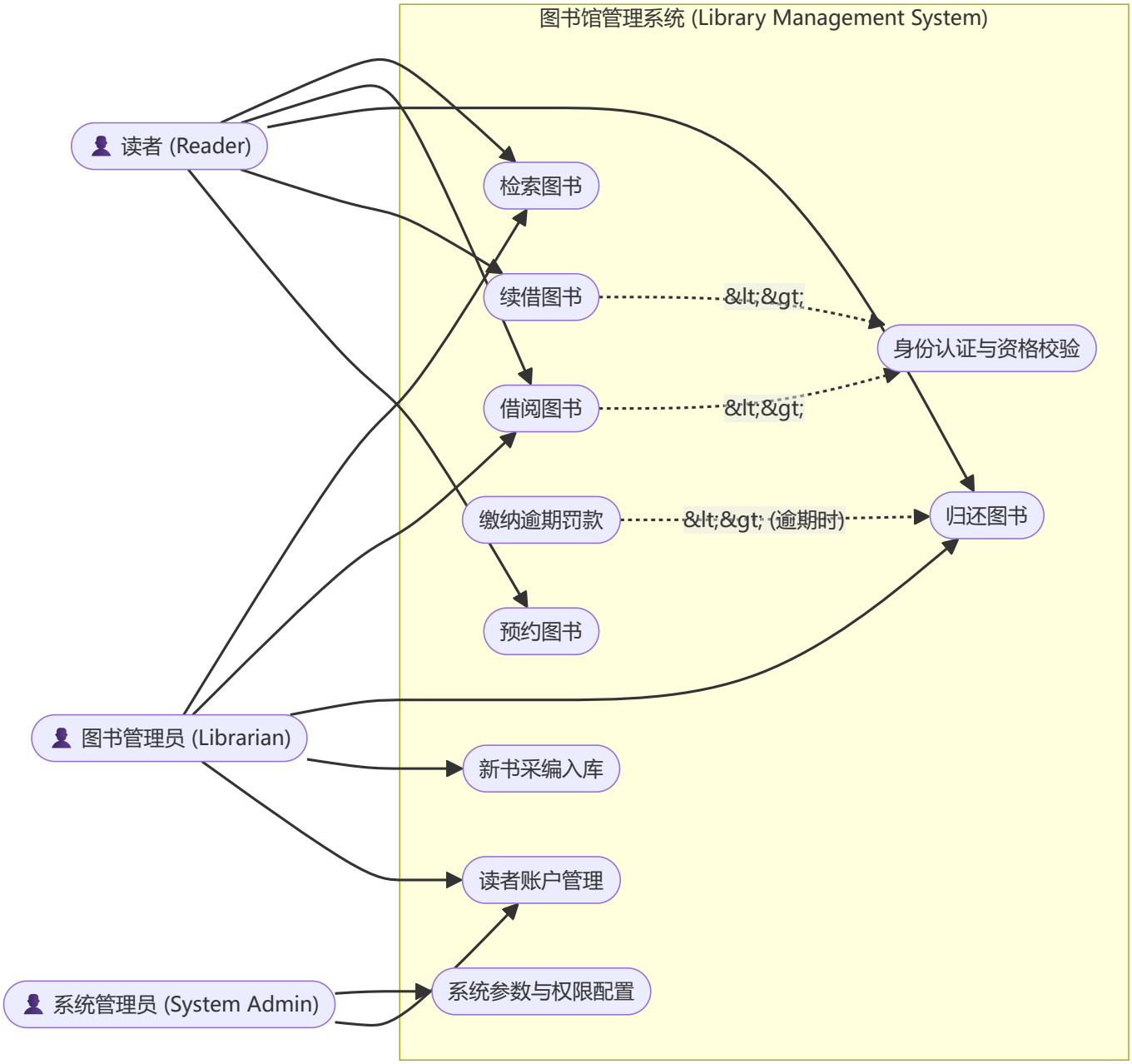
Ctrl -> BookSvc : 11. updateStatus(barcode, Borrowed)
activate BookSvc
BookSvc -> DB : 12. updateBookItemStatus()
deactivate BookSvc

Ctrl --> UI : 13. 返回借阅成功与应还日期
deactivate Ctrl
UI --> Reader : 14. 打印/显示借阅凭条
deactivate UI
@enduml
```

2. Use case diagram（用例图）

描述图书馆管理系统中**读者（Reader）**、**图书管理员（Librarian）**和系统管理员（System Admin）**三类参与者与系统核心用例之间的关系（包含 `<<include>>` 与 `<<extend>>` 关系）。

Mermaid 预览版



```
flowchart LR
    Reader(["👤 读者 (Reader)"])
    Librarian(["👤 图书管理员 (Librarian)"])
    Admin(["👤 系统管理员 (System Admin)"])

    subgraph LMS ["图书馆管理系统 (Library Management System)"]
        UC_Search(["检索图书"])
        UC_Borrow(["借阅图书"])
        UC_Return(["归还图书"])
        UC_Renew(["续借图书"])
        UC_Reserve(["预约图书"])
        UC_Auth(["身份认证与资格校验"])
        UC_Fine(["缴纳逾期罚款"])
        UC_Catalog(["新书采编入库"])
        UC_UserMgr(["读者账户管理"])
        UC_SysCfg(["系统参数与权限配置"])
    end

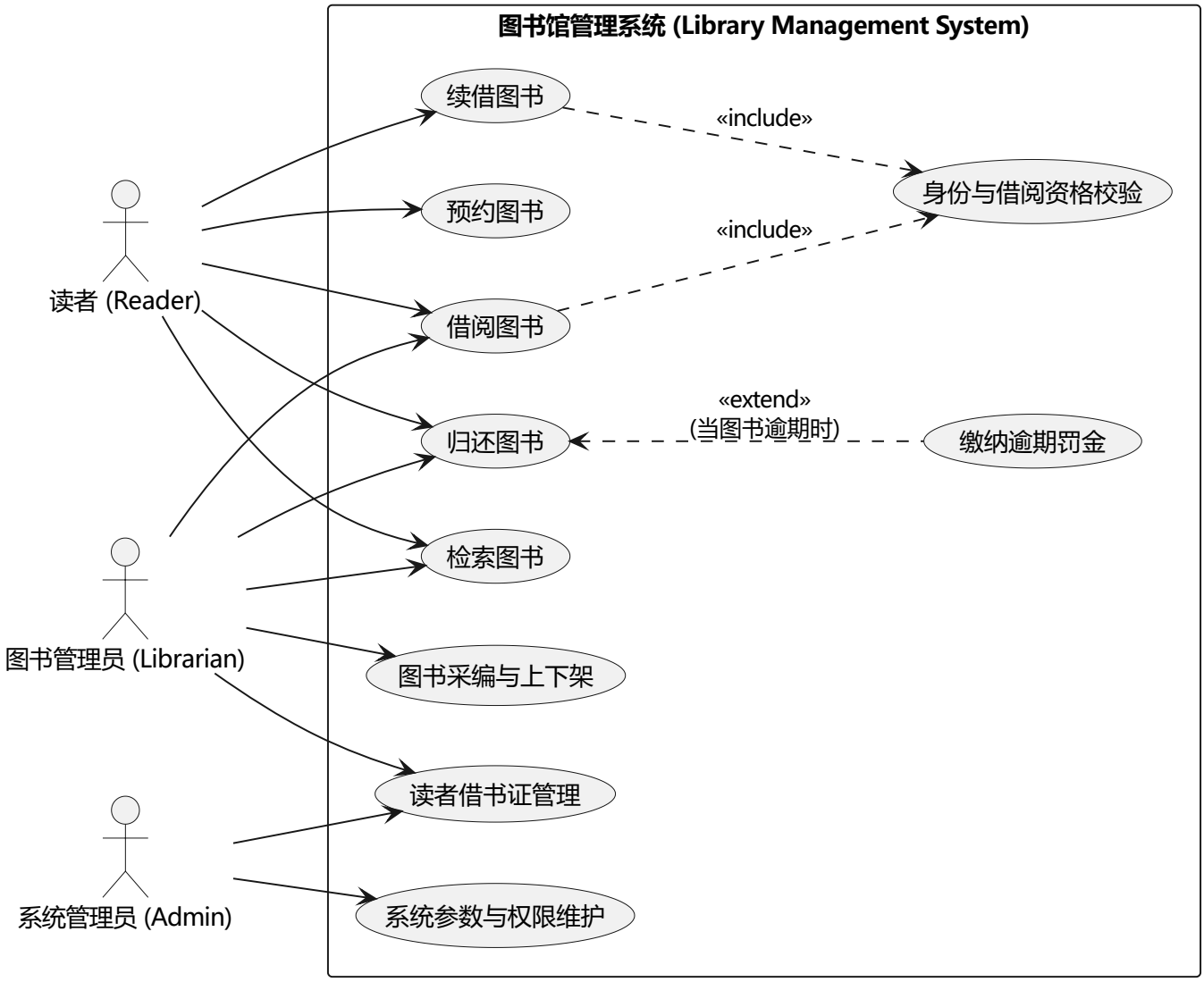
    Reader --> UC_Search
    Reader --> UC_Borrow
    Reader --> UC_Return
    Reader --> UC_Renew
    Reader --> UC_Reserve

    UC_Borrow -. "<<include>>" .-> UC_Auth
    UC_Renew -. "<<include>>" .-> UC_Auth
    UC_Fine -. "<<extend>> (逾期时)" .-> UC_Return

    Librarian --> UC_Search
    Librarian --> UC_Borrow
    Librarian --> UC_Return
    Librarian --> UC_Catalog
    Librarian --> UC_UserMgr

    Admin --> UC_SysCfg
    Admin --> UC_UserMgr
```

PlantUML 标准版


图书馆管理系统 (Library Management System)
续借图书
预约图书
借阅图书
归还图书
检索图书
图书采编与上下架
读者借书证管理
系统参数与权限维护
身份与借阅资格校验
缴纳逾期罚金
«include»
«include»
«extend»
(当图书逾期时)
读者 (Reader)
图书管理员 (Librarian)
系统管理员 (Admin)

```
@startuml
left to right direction
actor "读者 (Reader)" as Reader
actor "图书管理员 (Librarian)" as Librarian
actor "系统管理员 (Admin)" as Admin

rectangle "图书馆管理系统 (Library Management System)" {
    usecase "检索图书" as UC_Search
    usecase "借阅图书" as UC_Borrow
    usecase "归还图书" as UC_Return
    usecase "续借图书" as UC_Renew
    usecase "预约图书" as UC_Reserve
    usecase "身份与借阅资格校验" as UC_Auth
    usecase "缴纳逾期罚金" as UC_Fine
    usecase "图书采编与上下架" as UC_Catalog
    usecase "读者借书证管理" as UC_ReaderMgr
    usecase "系统参数与权限维护" as UC_SysCfg

    Reader --> UC_Search
    Reader --> UC_Borrow
    Reader --> UC_Return
    Reader --> UC_Renew
    Reader --> UC_Reserve

    Librarian --> UC_Search
    Librarian --> UC_Borrow
    Librarian --> UC_Return
    Librarian --> UC_Catalog
    Librarian --> UC_ReaderMgr

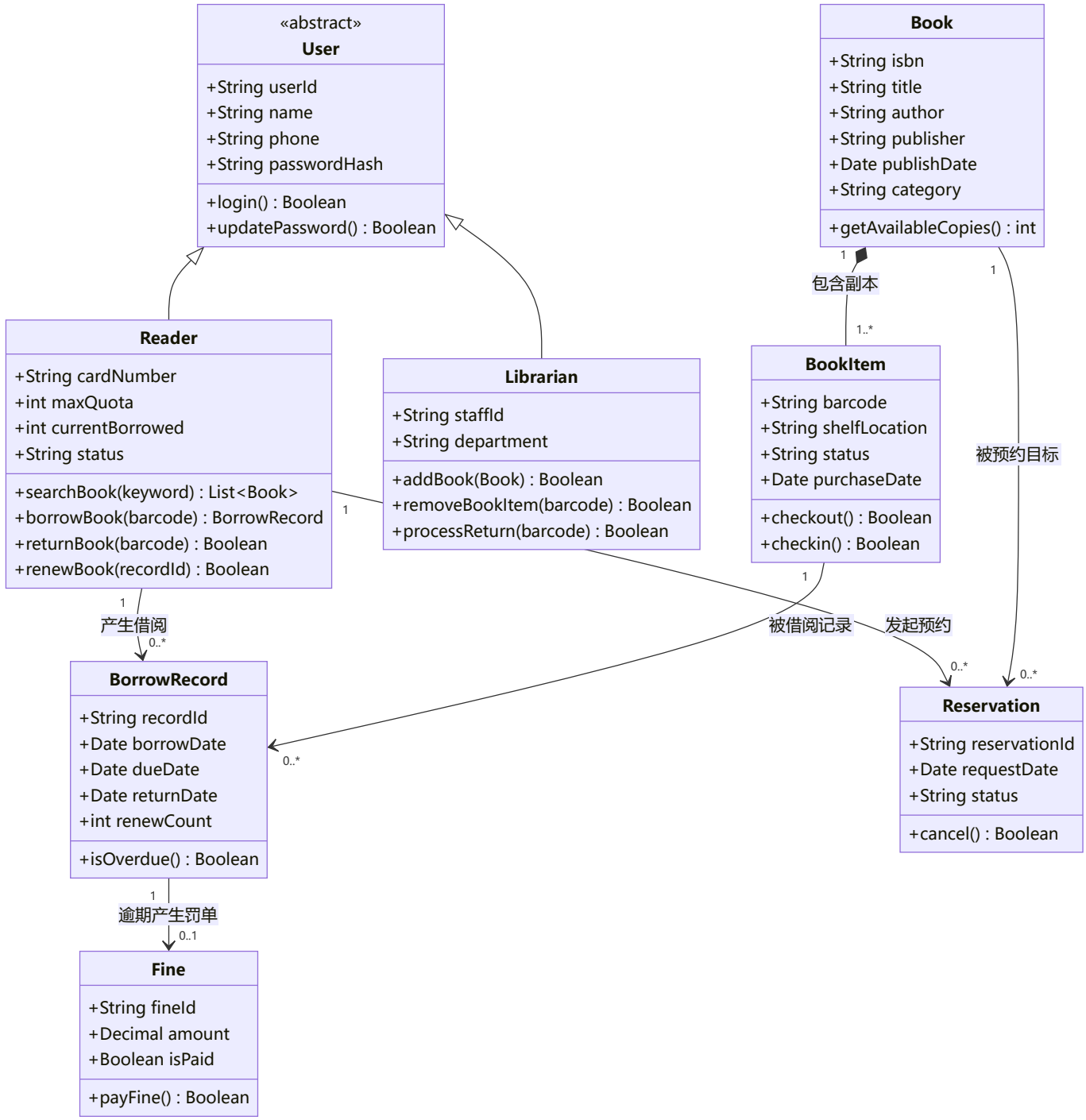
    Admin --> UC_SysCfg
    Admin --> UC_ReaderMgr

    UC_Borrow ..> UC_Auth : <<include>>
    UC_Renew ..> UC_Auth : <<include>>
    UC_Return <.. UC_Fine : <<extend>>\n(当图书逾期时)
}
@enduml
```

3. Class diagram（类图）

描述图书馆管理系统的核心领域类（`User`、`Reader`、`Librarian`、`Book`、`BookItem`、`BorrowRecord`、`Reservation`、`Fine`）及其继承、组合、关联与多重性关系。

Mermaid 预览版



```
classDiagram
    class User {
        <<abstract>>
        +String userId
        +String name
        +String phone
        +String passwordHash
        +login() Boolean
        +updatePassword() Boolean
    }

    class Reader {
        +String cardNumber
        +int maxQuota
        +int currentBorrowed
        +String status
        +searchBook(keyword) List~Book~
        +borrowBook(barcode) BorrowRecord
        +returnBook(barcode) Boolean
        +renewBook(recordId) Boolean
    }

    class Librarian {
        +String staffId
        +String department
        +addBook(Book) Boolean
        +removeBookItem(barcode) Boolean
        +processReturn(barcode) Boolean
    }

    class Book {
        +String isbn
        +String title
        +String author
        +String publisher
        +Date publishDate
        +String category
        +getAvailableCopies() int
    }

    class BookItem {
        +String barcode
        +String shelfLocation
        +String status
        +Date purchaseDate
        +checkout() Boolean
        +checkin() Boolean
    }

    class BorrowRecord {
        +String recordId
        +Date borrowDate
        +Date dueDate
        +Date returnDate
        +int renewCount
        +isOverdue() Boolean
    }

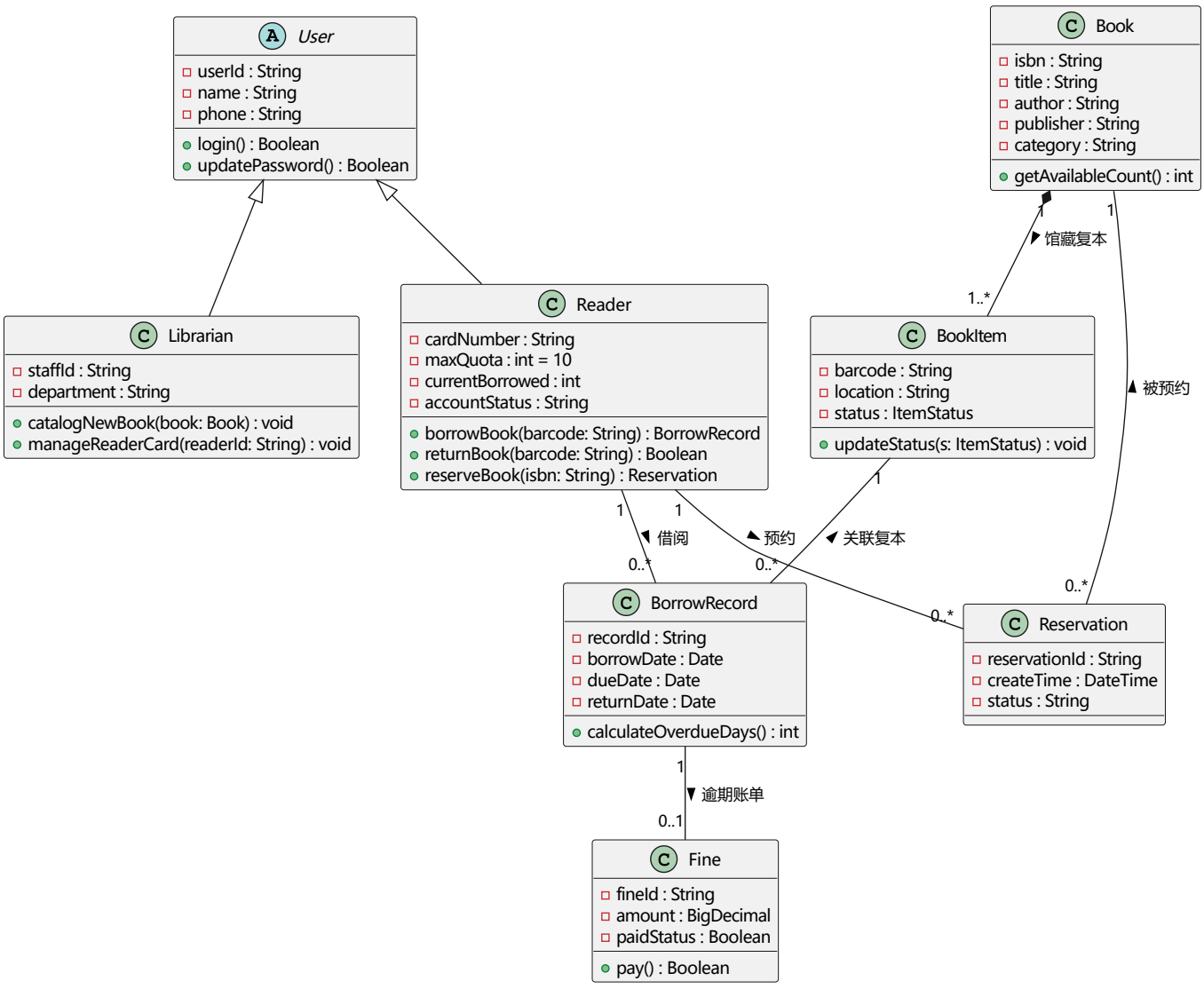
    class Reservation {
        +String reservationId
        +Date requestDate
```

```
        +String status
        +cancel() Boolean
    }

    class Fine {
        +String fineId
        +Decimal amount
        +Boolean isPaid
        +payFine() Boolean
    }

    User <|-- Reader
    User <|-- Librarian
    Book "1" *-- "1..*" BookItem : 包含副本
    Reader "1" --> "0..*" BorrowRecord : 产生借阅
    BookItem "1" --> "0..*" BorrowRecord : 被借阅记录
    Reader "1" --> "0..*" Reservation : 发起预约
    Book "1" --> "0..*" Reservation : 被预约目标
    BorrowRecord "1" --> "0..1" Fine : 逾期产生罚单
```

PlantUML 标准版



```
@startuml
abstract class User {
    - userId : String
    - name : String
    - phone : String
    + login() : Boolean
    + updatePassword() : Boolean
}

class Reader {
    - cardNumber : String
    - maxQuota : int = 10
    - currentBorrowed : int
    - accountStatus : String
    + borrowBook(barcode: String) : BorrowRecord
    + returnBook(barcode: String) : Boolean
    + reserveBook(isbn: String) : Reservation
}

class Librarian {
    - staffId : String
    - department : String
    + catalogNewBook(book: Book) : void
    + manageReaderCard(readerId: String) : void
}

class Book {
    - isbn : String
    - title : String
    - author : String
    - publisher : String
    - category : String
    + getAvailableCount() : int
}

class BookItem {
    - barcode : String
    - location : String
    - status : ItemStatus
    + updateStatus(s: ItemStatus) : void
}

class BorrowRecord {
    - recordId : String
    - borrowDate : Date
    - dueDate : Date
    - returnDate : Date
    + calculateOverdueDays() : int
}

class Reservation {
    - reservationId : String
    - createTime : DateTime
    - status : String
}

class Fine {
    - fineId : String
    - amount : BigDecimal
    - paidStatus : Boolean
    + pay() : Boolean
```

```
}

User <|-- Reader
User <|-- Librarian
Book "1" *-- "1..*" BookItem : 馆藏复本 >
Reader "1" -- "0..*" BorrowRecord : 借阅 >
BookItem "1" -- "0..*" BorrowRecord : 关联复本 <
Reader "1" -- "0..*" Reservation : 预约 >
Book "1" -- "0..*" Reservation : 被预约 <
BorrowRecord "1" -- "0..1" Fine : 逾期账单 >
@enduml
```

4. Object diagram（对象图）

展示系统在**2026年10月30日读者王同学借阅《软件工程导论》第1号复本**这一具体时刻的运行时对象实例及对象链快照。

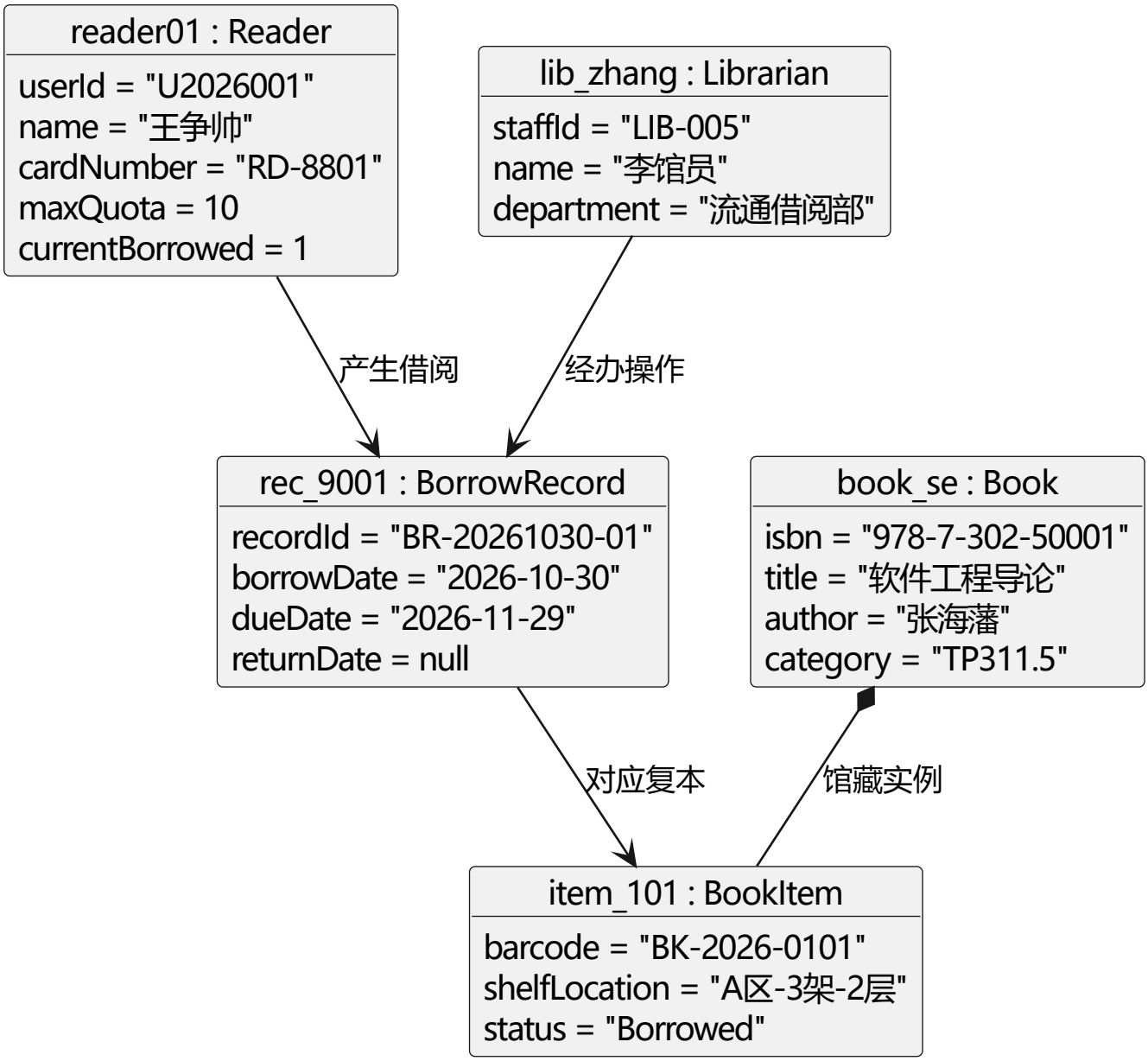
Mermaid 预览版

> （此段 Mermaid 代码存在语法错误，无法渲染；同一图的 PlantUML 版本见下方）

```
flowchart TB
    subgraph Objects ["2026-10-30 借阅快照对象图 (Object Diagram)"]
        r1["<b><u>reader01 : Reader</u></b><br/>userId = 'U2026001'<br/>name = '王争帅'<br/>cardNumber = 'RD-8801'<br/>maxQuota = 10<br/>currentBorrowed = 1"]
        b1["<b><u>book_se : Book</u></b><br/>isbn = '978-7-302-50001'<br/>title = '软件工程导论'<br/>author = '张海藩'<br/>category = 'TP311.5'"]
        item1["<b><u>item_101 : BookItem</u></b><br/>barcode = 'BK-2026-0101'<br/>shelfLocation = 'A区-3架-2层'<br/>status = 'Borrowed'"]
        rec1["<b><u>rec_9001 : BorrowRecord</u></b><br/>recordId = 'BR-20261030-01'<br/>borrowDate = '2026-10-30'<br/>dueDate = '2026-11-29'<br/>returnDate = null"]
        lib1["<b><u>lib_zhang : Librarian</u></b><br/>staffId = 'LIB-005'<br/>name = '李馆员'<br/>department = '流通借阅部'"]
    end

    r1 -- "持有借阅记录" --> rec1
    rec1 -- "关联馆藏复本" --> item1
    b1 *-- "属于书目" --- item1
    lib1 -- "经办审核" --> rec1
```

PlantUML 标准版


reader01 : Reader
userId = "U2026001"
name = "王争帅"
cardNumber = "RD-8801"
maxQuota = 10
currentBorrowed = 1
lib_zhang : Librarian
staffId = "LIB-005"
name = "李馆员"
department = "流通借阅部"
产生借阅
经办操作
rec_9001 : BorrowRecord
recordId = "BR-20261030-01"
borrowDate = "2026-10-30"
dueDate = "2026-11-29"
returnDate = null
book_se : Book
isbn = "978-7-302-50001"
title = "软件工程导论"
author = "张海藩"
category = "TP311.5"
对应复本
馆藏实例
item_101 : BookItem
barcode = "BK-2026-0101"
shelfLocation = "A区-3架-2层"
status = "Borrowed"

```
@startuml
object "reader01 : Reader" as r1 {
    userId = "U2026001"
    name = "王争帅"
    cardNumber = "RD-8801"
    maxQuota = 10
    currentBorrowed = 1
}

object "book_se : Book" as b1 {
    isbn = "978-7-302-50001"
    title = "软件工程导论"
    author = "张海藩"
    category = "TP311.5"
}

object "item_101 : BookItem" as item1 {
    barcode = "BK-2026-0101"
    shelfLocation = "A区-3架-2层"
    status = "Borrowed"
}

object "rec_9001 : BorrowRecord" as rec1 {
    recordId = "BR-20261030-01"
    borrowDate = "2026-10-30"
    dueDate = "2026-11-29"
    returnDate = null
}

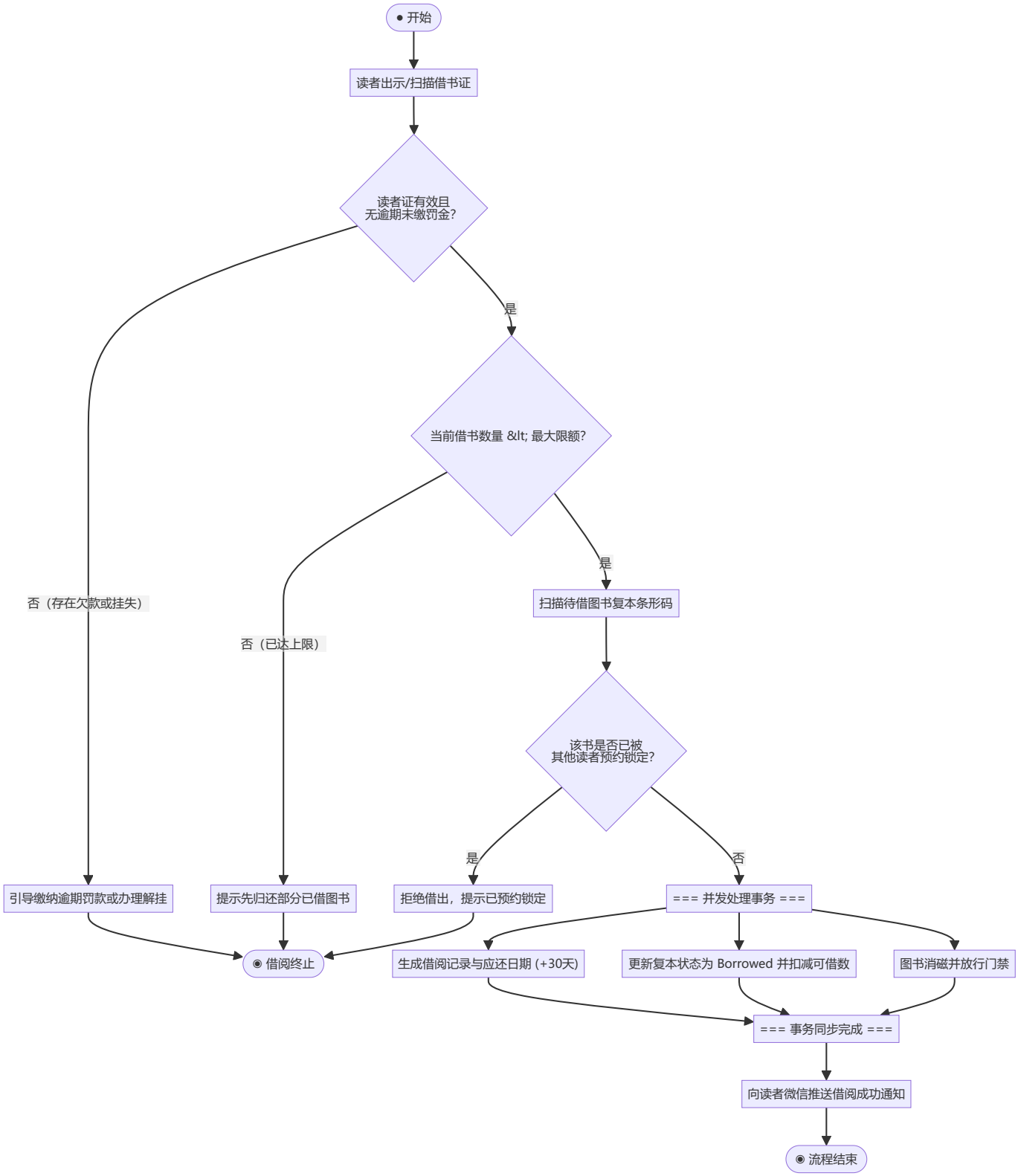
object "lib_zhang : Librarian" as lib1 {
    staffId = "LIB-005"
    name = "李馆员"
    department = "流通借阅部"
}

r1 --> rec1 : 产生借阅
rec1 --> item1 : 对应复本
b1 *-- item1 : 馆藏实例
lib1 --> rec1 : 经办操作
@enduml
```

5. Activity diagram（活动图 - 新语法）

描述读者在图书馆进行**借书与资格审查、归还与逾期处理**的完整业务活动流（采用标准新语法 `start / if / fork / stop`）。

Mermaid 预览版



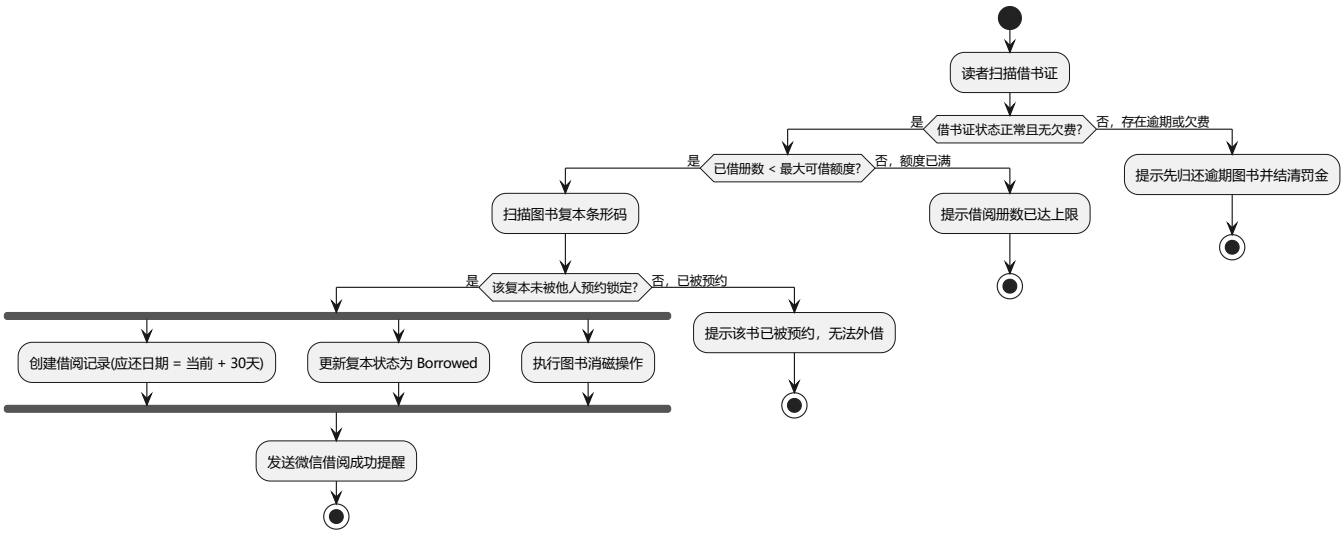
```mermaid
flowchart TD
    Start(["● 开始"]) --> ScanCard["读者出示/扫描借书证"]
    ScanCard --> CheckCard{"读者证有效且<br/>无逾期未缴罚金？"}
    CheckCard -- "否（存在欠款或挂失）" --> PayOrUnlock["引导缴纳逾期罚款或办理解挂"]
    PayOrUnlock --> EndFail(["◉ 借阅终止"])

    CheckCard -- "是" --> CheckQuota{"当前借书数量 < 最大限额？"}
    CheckQuota -- "否（已达上限）" --> PromptReturn["提示先归还部分已借图书"]
    PromptReturn --> EndFail

    CheckQuota -- "是" --> ScanBook["扫描待借图书复本条形码"]
    ScanBook --> CheckReserved{"该书是否已被<br/>其他读者预约锁定？"}
    CheckReserved -- "是" --> RejectBorrow["拒绝借出，提示已预约锁定"]
    RejectBorrow --> EndFail

    CheckReserved -- "否" --> ForkStart["=== 并发处理事务 ==="]
    ForkStart --> GenRecord["生成借阅记录与应还日期（+30天）"]
    ForkStart --> UpdateStock["更新复本状态为 Borrowed 并扣减可借数"]
    ForkStart --> DeMagnetize["图书消磁并放行门禁"]
    GenRecord --> ForkJoin["=== 事务同步完成 ==="]
    UpdateStock --> ForkJoin
    DeMagnetize --> ForkJoin
    ForkJoin --> SendNotice["向读者微信推送借阅成功通知"]
    SendNotice --> EndSuccess(["◉ 流程结束"])
```

PlantUML 标准版（新语法）

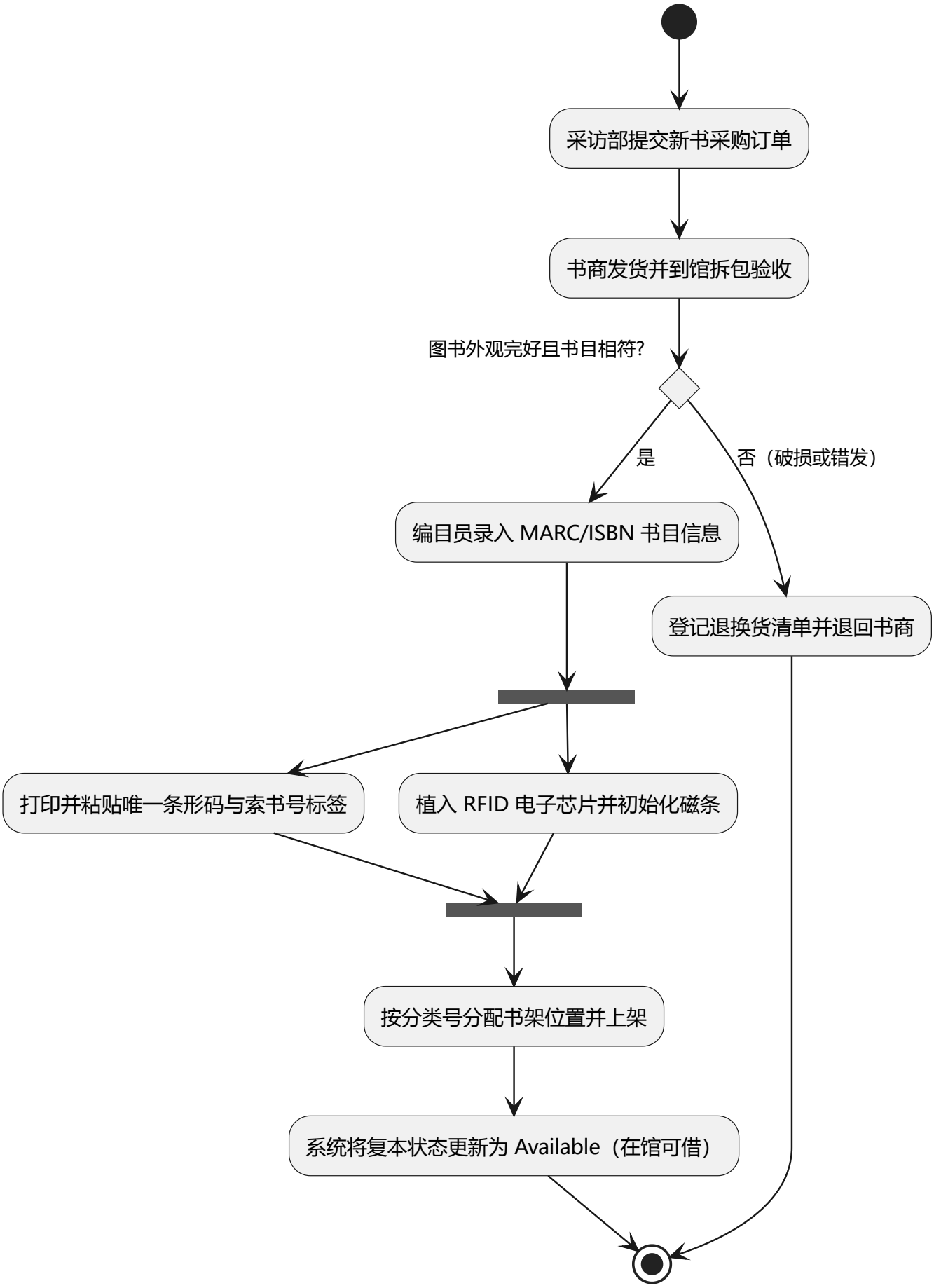


```
@startuml
start
:读者扫描借书证;
if (借书证状态正常且无欠费?) then (是)
    if (已借册数 < 最大可借额度?) then (是)
        :扫描图书复本条形码;
        if (该复本未被他人预约锁定?) then (是)
            fork
                :创建借阅记录(应还日期 = 当前 + 30天);
            fork again
                :更新复本状态为 Borrowed;
            fork again
                :执行图书消磁操作;
            end fork
            :发送微信借阅成功提醒;
            stop
        else (否，已被预约)
            :提示该书已被预约，无法外借;
            stop
        endif
    else (否，额度已满)
        :提示借阅册数已达上限;
        stop
    endif
else (否，存在逾期或欠费)
    :提示先归还逾期图书并结清罚金;
    stop
endif
@enduml
```

6. Activity diagram - Legacy syntax（活动图 - 旧语法）

对应实验要求第 10 行的 `Legacy syntax`（PlantUML 传统 `(*) -->` 活动图语法），描述**图书馆新书采购、验收编目、贴码上架及归还质检**流程。

PlantUML 标准版（Legacy syntax `(*) -->`）


采访部提交新书采购订单
书商发货并到馆拆包验收
图书外观完好且书目相符?
是
否（破损或错发）
编目员录入 MARC/ISBN 书目信息
登记退换货清单并退回书商
打印并粘贴唯一条形码与索书号标签
植入 RFID 电子芯片并初始化磁条
按分类号分配书架位置并上架
系统将复本状态更新为 Available（在馆可借）

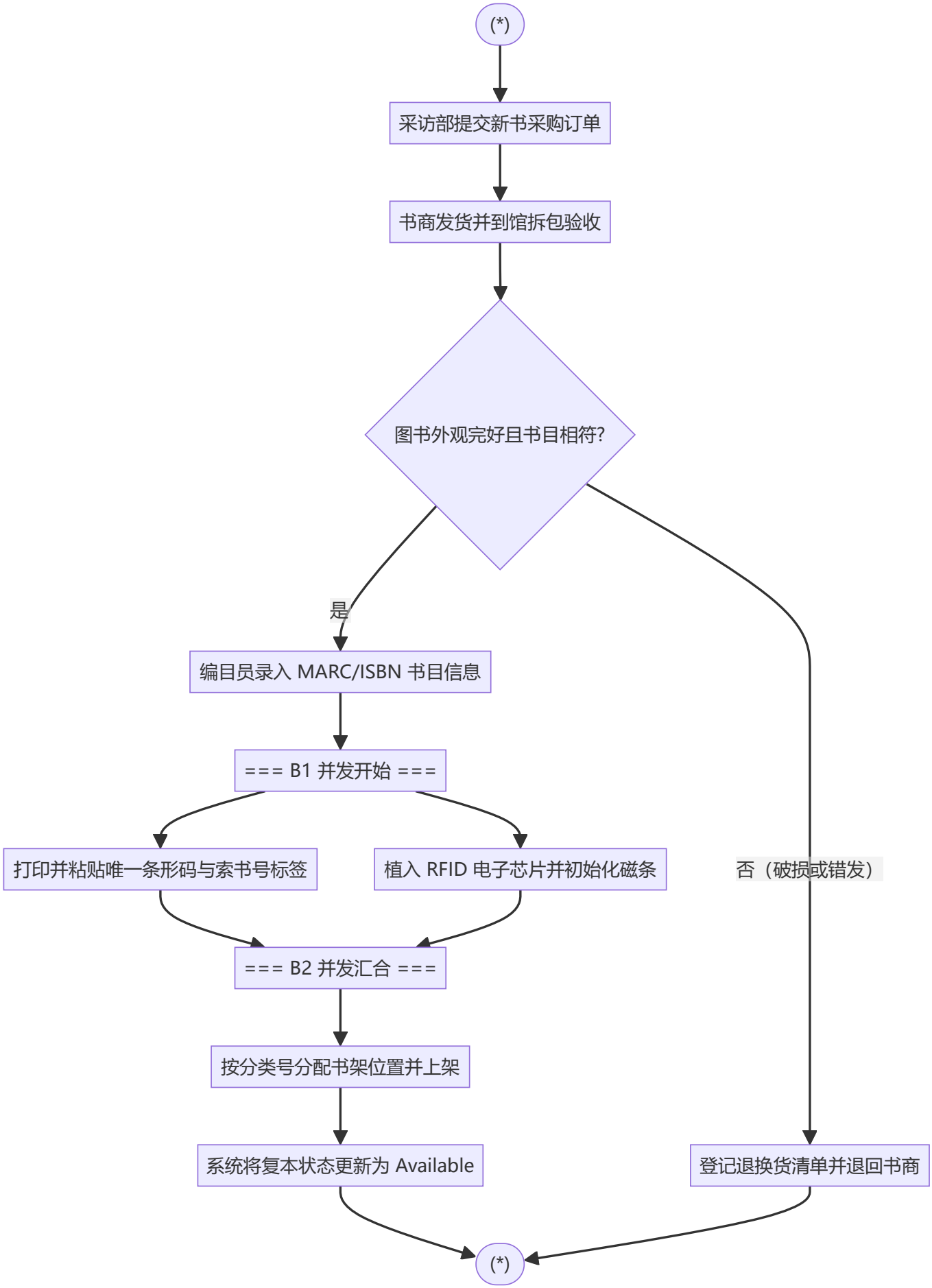
```
@startuml
(*) --> "采访部提交新书采购订单"
--> "书商发货并到馆拆包验收"

if "图书外观完好且书目相符?" then
    --> [是] "编目员录入 MARC/ISBN 书目信息"
    --> === B1 ===
    --> "打印并粘贴唯一条形码与索书号标签"
    --> === B2 ===

    === B1 === --> "植入 RFID 电子芯片并初始化磁条"
    --> === B2 ===

    === B2 === --> "按分类号分配书架位置并上架"
    --> "系统将复本状态更新为 Available（在馆可借）"
    --> (*)
else
    --> [否（破损或错发）] "登记退换货清单并退回书商"
    --> (*)
endif
@enduml
```

Mermaid 预览版（对应旧语法流程）

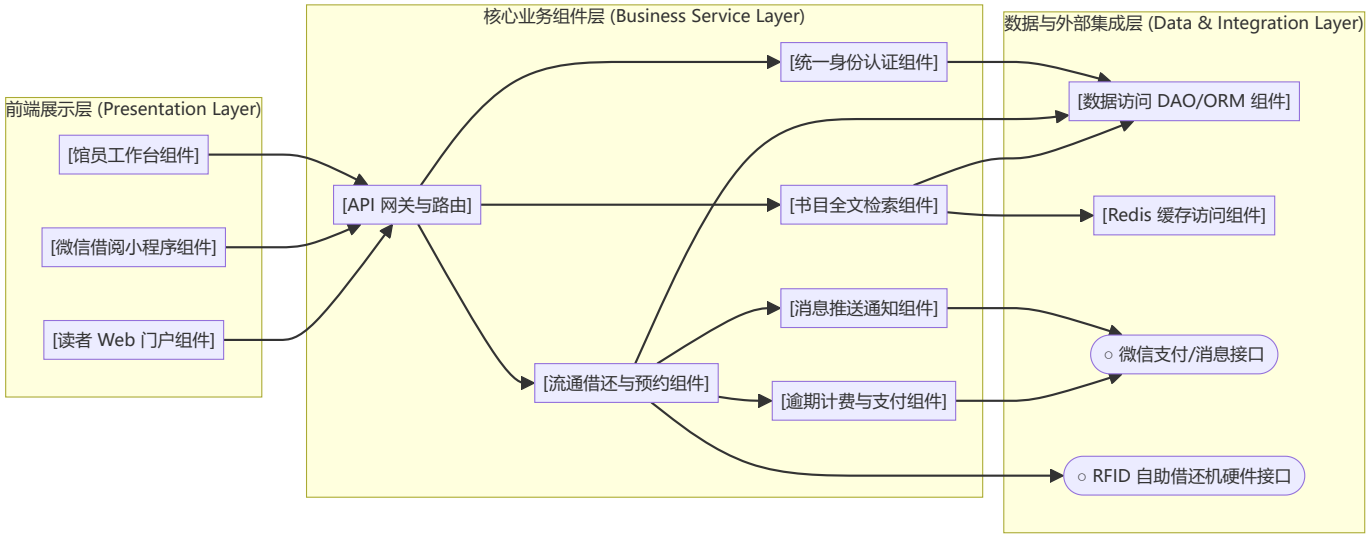

(*)
采访部提交新书采购订单
书商发货并到馆拆包验收
图书外观完好且书目相符?
是
否（破损或错发）
编目员录入 MARC/ISBN 书目信息
=== B1 并发开始 ===
打印并粘贴唯一条形码与索书号标签
植入 RFID 电子芯片并初始化磁条
=== B2 并发汇合 ===
按分类号分配书架位置并上架
系统将复本状态更新为 Available
登记退换货清单并退回书商
(*)

```mermaid
flowchart TD
    Start(["(*)"]) --> Order["采访部提交新书采购订单"]
    Order --> Inspect["书商发货并到馆拆包验收"]
    Inspect --> Cond{"图书外观完好且书目相符?"}
    Cond -- "是" --> Catalog["编目员录入 MARC/ISBN 书目信息"]
    Catalog --> B1["=== B1 并发开始 ==="]
    B1 --> Barcode["打印并粘贴唯一条形码与索书号标签"]
    B1 --> RFID["植入 RFID 电子芯片并初始化磁条"]
    Barcode --> B2["=== B2 并发汇合 ==="]
    RFID --> B2
    B2 --> Shelf["按分类号分配书架位置并上架"]
    Shelf --> Ready["系统将复本状态更新为 Available"]
    Ready --> EndNode(["(*)"])
    Cond -- "否（破损或错发）" --> ReturnVendor["登记退换货清单并退回书商"]
    ReturnVendor --> EndNode
```

7. Component diagram（组件图 / 构件图）

展示图书馆管理系统的软件模块划分：前端表现层组件、核心业务微服务/模块组件、外部接口组件及数据持久化组件之间的接口调用依赖。

Mermaid 预览版



```mermaid
flowchart LR
    subgraph ClientLayer ["前端展示层 (Presentation Layer)"]
        WebPortal["[读者 Web 门户组件]"]
        MiniApp["[微信借阅小程序组件]"]
        AdminConsole["[馆员工作台组件]"]
    end

    subgraph ServiceLayer ["核心业务组件层 (Business Service Layer)"]
        APIGateway["[API 网关与路由]"]
        AuthComp["[统一身份认证组件]"]
        SearchComp["[书目全文检索组件]"]
        CirculationComp["[流通借还与预约组件]"]
        FineComp["[逾期计费与支付组件]"]
        NoticeComp["[消息推送通知组件]"]
    end

    subgraph DataLayer ["数据与外部集成层 (Data & Integration Layer)"]
        ORMComp["[数据访问 DAO/ORM 组件]"]
        CacheComp["[Redis 缓存访问组件]"]
        WeChatAPI(["○ 微信支付/消息接口"])
        RFIDHardware(["○ RFID 自助借还机硬件接口"])
    end

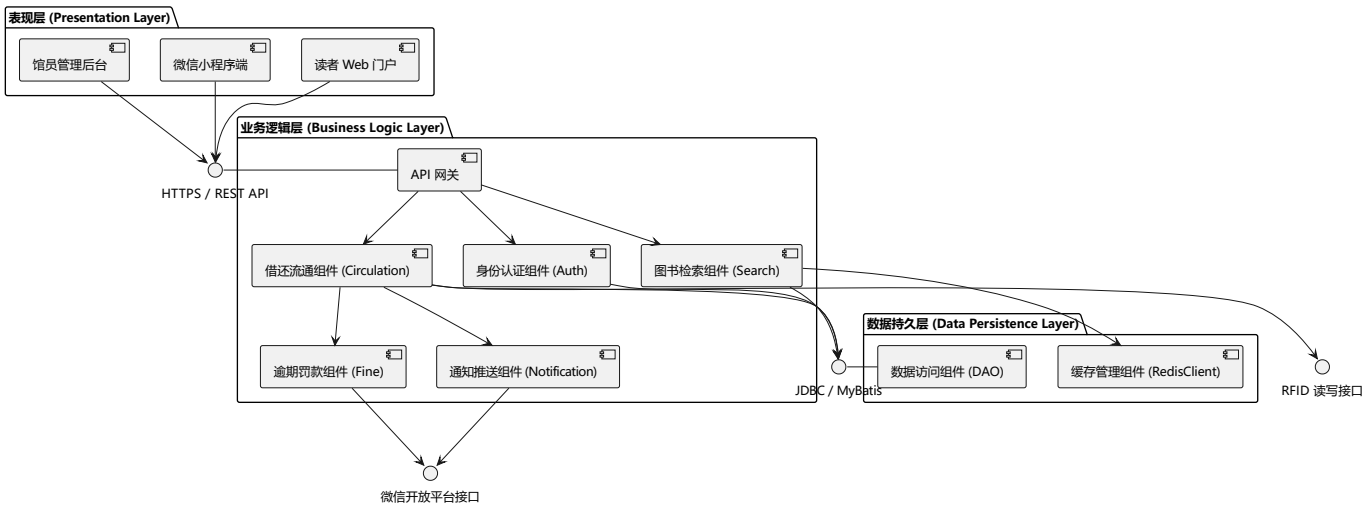
    WebPortal --> APIGateway
    MiniApp --> APIGateway
    AdminConsole --> APIGateway

    APIGateway --> AuthComp
    APIGateway --> SearchComp
    APIGateway --> CirculationComp

    CirculationComp --> FineComp
    CirculationComp --> NoticeComp
    CirculationComp --> RFIDHardware

    SearchComp --> CacheComp
    SearchComp --> ORMComp
    CirculationComp --> ORMComp
    AuthComp --> ORMComp
    FineComp --> WeChatAPI
    NoticeComp --> WeChatAPI
```

PlantUML 标准版



```
@startuml
package "表现层 (Presentation Layer)" {
    [读者 Web 门户] as WebUI
    [微信小程序端] as MiniApp
    [馆员管理后台] as AdminUI
}

package "业务逻辑层 (Business Logic Layer)" {
    [API 网关] as Gateway
    [身份认证组件 (Auth)] as Auth
    [图书检索组件 (Search)] as Search
    [借还流通组件 (Circulation)] as Circ
    [逾期罚款组件 (Fine)] as Fine
    [通知推送组件 (Notification)] as Notify
}

package "数据持久层 (Data Persistence Layer)" {
    [数据访问组件 (DAO)] as DAO
    [缓存管理组件 (RedisClient)] as Cache
}

interface "HTTPS / REST API" as IRest
interface "JDBC / MyBatis" as IDB
interface "RFID 读写接口" as IRFID
interface "微信开放平台接口" as IWx

WebUI --> IRest
MiniApp --> IRest
AdminUI --> IRest
IRest - [Gateway]

Gateway --> Auth
Gateway --> Search
Gateway --> Circ

Circ --> Fine
Circ --> Notify
Circ --> IRFID

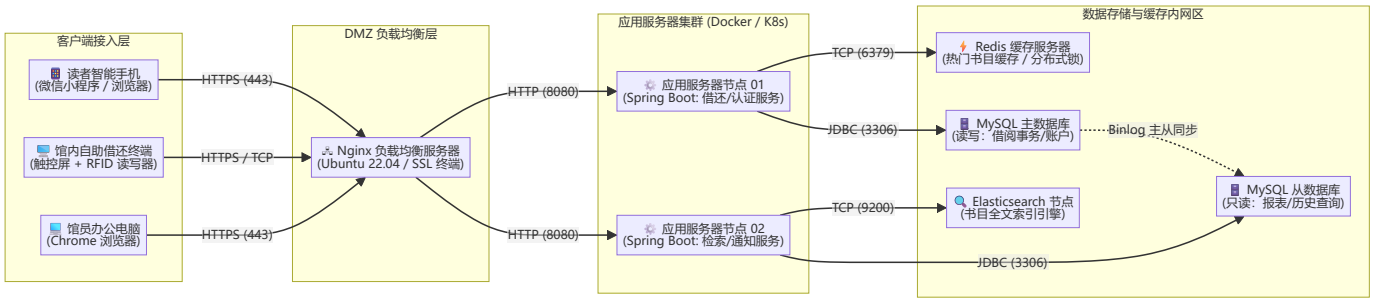
Search --> Cache
Search --> IDB
Circ --> IDB
Auth --> IDB
IDB - [DAO]

Fine --> IWx
Notify --> IWx
@enduml
```

8. Deployment diagram（部署图）

描述图书馆管理系统的物理硬件拓扑、运行容器节点、网络协议以及构件在物理服务器上的部署分布。

Mermaid 预览版




```mermaid
flowchart LR
    subgraph ClientNodes ["客户端接入层"]
        ReaderPhone["📱 读者智能手机<br/>(微信小程序 / 浏览器)"]
        SelfMachine["🖥️ 馆内自助借还终端<br/>(触控屏 + RFID 读写器)"]
        StaffPC["💻 馆员办公电脑<br/>(Chrome 浏览器)"]
    end

    subgraph DMZ ["DMZ 负载均衡层"]
        NginxNode["🖧 Nginx 负载均衡服务器<br/>(Ubuntu 22.04 / SSL 终端)"]
    end

    subgraph AppCluster ["应用服务器集群 (Docker / K8s)"]
        AppSrv1["⚙️ 应用服务器节点 01<br/>(Spring Boot: 借还/认证服务)"]
        AppSrv2["⚙️ 应用服务器节点 02<br/>(Spring Boot: 检索/通知服务)"]
    end

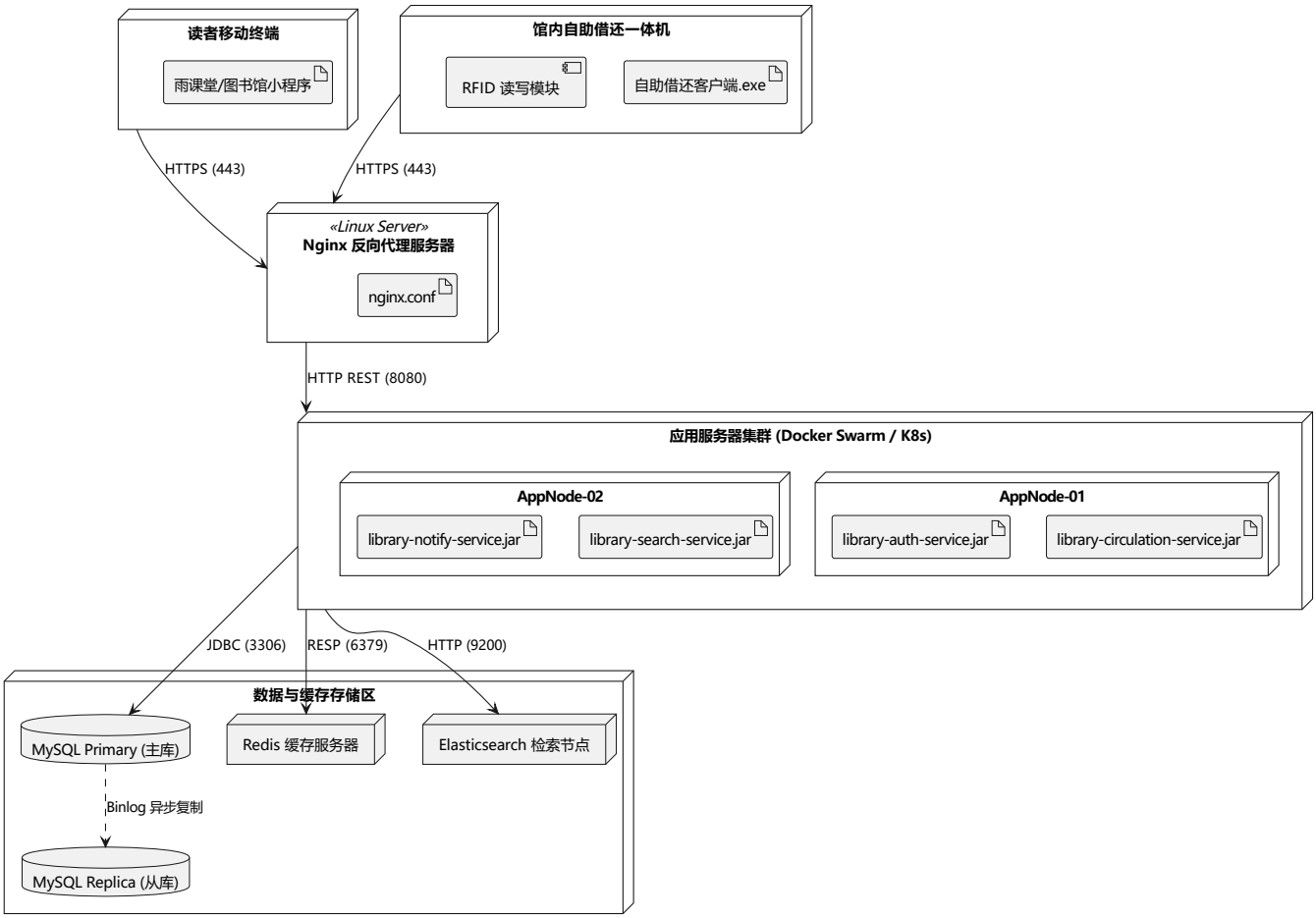
    subgraph StorageCluster ["数据存储与缓存内网区"]
        RedisNode["⚡ Redis 缓存服务器<br/>(热门书目缓存 / 分布式锁)"]
        ESNode["🔍 Elasticsearch 节点<br/>(书目全文索引引擎)"]
        MySQLMaster["🗄️ MySQL 主数据库<br/>(读写：借阅事务/账户)"]
        MySQLSlave["🗄️ MySQL 从数据库<br/>(只读：报表/历史查询)"]
    end

    ReaderPhone -- "HTTPS (443)" --> NginxNode
    SelfMachine -- "HTTPS / TCP" --> NginxNode
    StaffPC -- "HTTPS (443)" --> NginxNode

    NginxNode -- "HTTP (8080)" --> AppSrv1
    NginxNode -- "HTTP (8080)" --> AppSrv2

    AppSrv1 -- "TCP (6379)" --> RedisNode
    AppSrv2 -- "TCP (9200)" --> ESNode
    AppSrv1 -- "JDBC (3306)" --> MySQLMaster
    AppSrv2 -- "JDBC (3306)" --> MySQLSlave
    MySQLMaster -. "Binlog 主从同步" .-> MySQLSlave
```

PlantUML 标准版


读者移动终端
雨课堂/图书馆小程序
馆内自助借还一体机
RFID 读写模块
自助借还客户端.exe
HTTPS (443)
HTTPS (443)
«Linux Server»
Nginx 反向代理服务器
nginx.conf
HTTP REST (8080)
应用服务器集群 (Docker Swarm / K8s)
AppNode-02
library-notify-service.jar
library-search-service.jar
AppNode-01
library-auth-service.jar
library-circulation-service.jar
JDBC (3306)
RESP (6379)
HTTP (9200)
数据与缓存存储区
MySQL Primary (主库)
Redis 缓存服务器
Elasticsearch 检索节点
Binlog 异步复制
MySQL Replica (从库)

```
@startuml
node "读者移动终端" as Phone {
    artifact "雨课堂/图书馆小程序" as MiniProgram
}

node "馆内自助借还一体机" as Kiosk {
    artifact "自助借还客户端.exe" as KioskApp
    component "RFID 读写模块" as RFID
}

node "Nginx 反向代理服务器" as LB <<Linux Server>> {
    artifact "nginx.conf" as Nginx
}

node "应用服务器集群 (Docker Swarm / K8s)" as AppCluster {
    node "AppNode-01" {
        artifact "library-circulation-service.jar" as CircJar
        artifact "library-auth-service.jar" as AuthJar
    }
    node "AppNode-02" {
        artifact "library-search-service.jar" as SearchJar
        artifact "library-notify-service.jar" as NotifyJar
    }
}

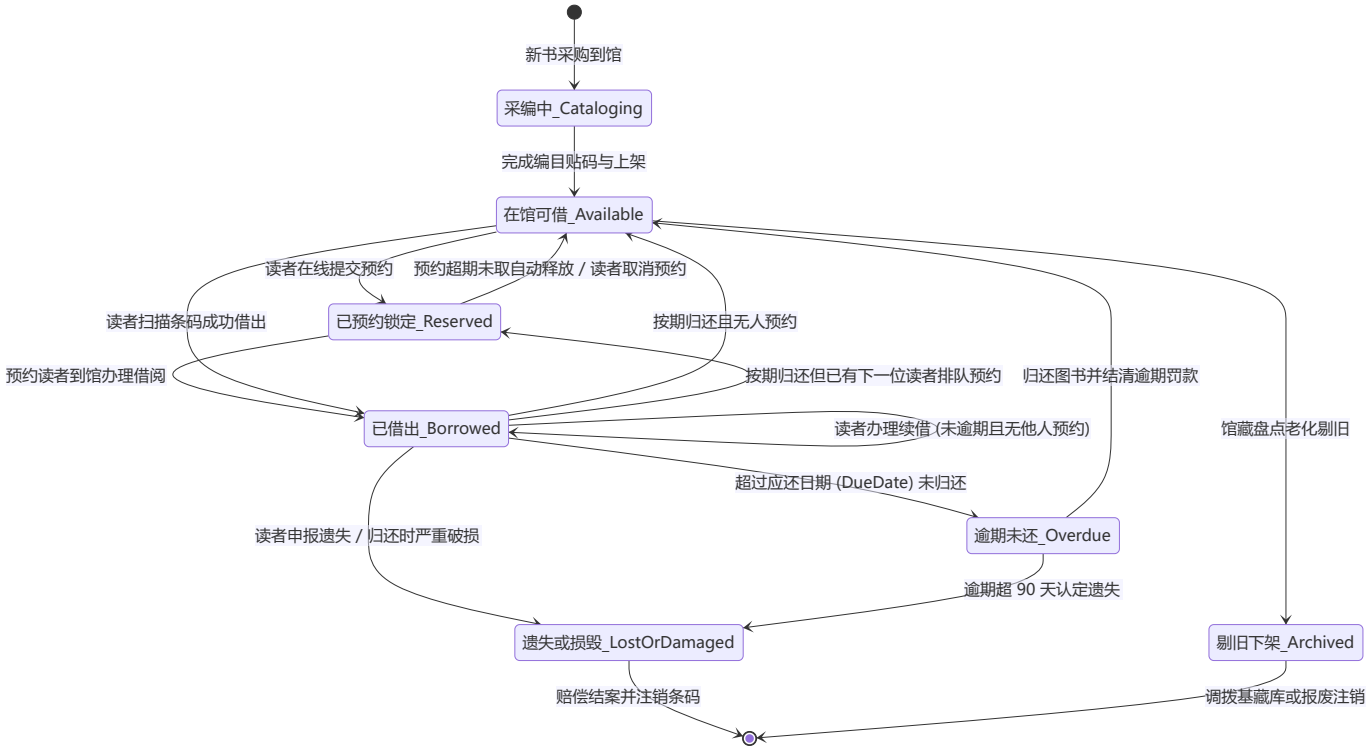
node "数据与缓存存储区" as DataZone {
    database "MySQL Primary (主库)" as DBMaster
    database "MySQL Replica (从库)" as DBSlave
    node "Redis 缓存服务器" as Redis
    node "Elasticsearch 检索节点" as ES
}

Phone --> LB : HTTPS (443)
Kiosk --> LB : HTTPS (443)
LB --> AppCluster : HTTP REST (8080)
AppCluster --> Redis : RESP (6379)
AppCluster --> ES : HTTP (9200)
AppCluster --> DBMaster : JDBC (3306)
DBMaster ..> DBSlave : Binlog 异步复制
@enduml
```

9. State diagram（状态图）

描述图书馆中**单本实体图书复本（`BookItem`）**从采购编目到外借、续借、逾期、归还乃至挂失或报废的完整状态生命周期转换。

Mermaid 预览版




```mermaid
stateDiagram-v2
    [*] --> 采编中_Cataloging : 新书采购到馆

    采编中_Cataloging --> 在馆可借_Available : 完成编目贴码与上架

    在馆可借_Available --> 已预约锁定_Reserved : 读者在线提交预约
    已预约锁定_Reserved --> 在馆可借_Available : 预约超期未取自动释放 / 读者取消预约
    已预约锁定_Reserved --> 已借出_Borrowed : 预约读者到馆办理借阅

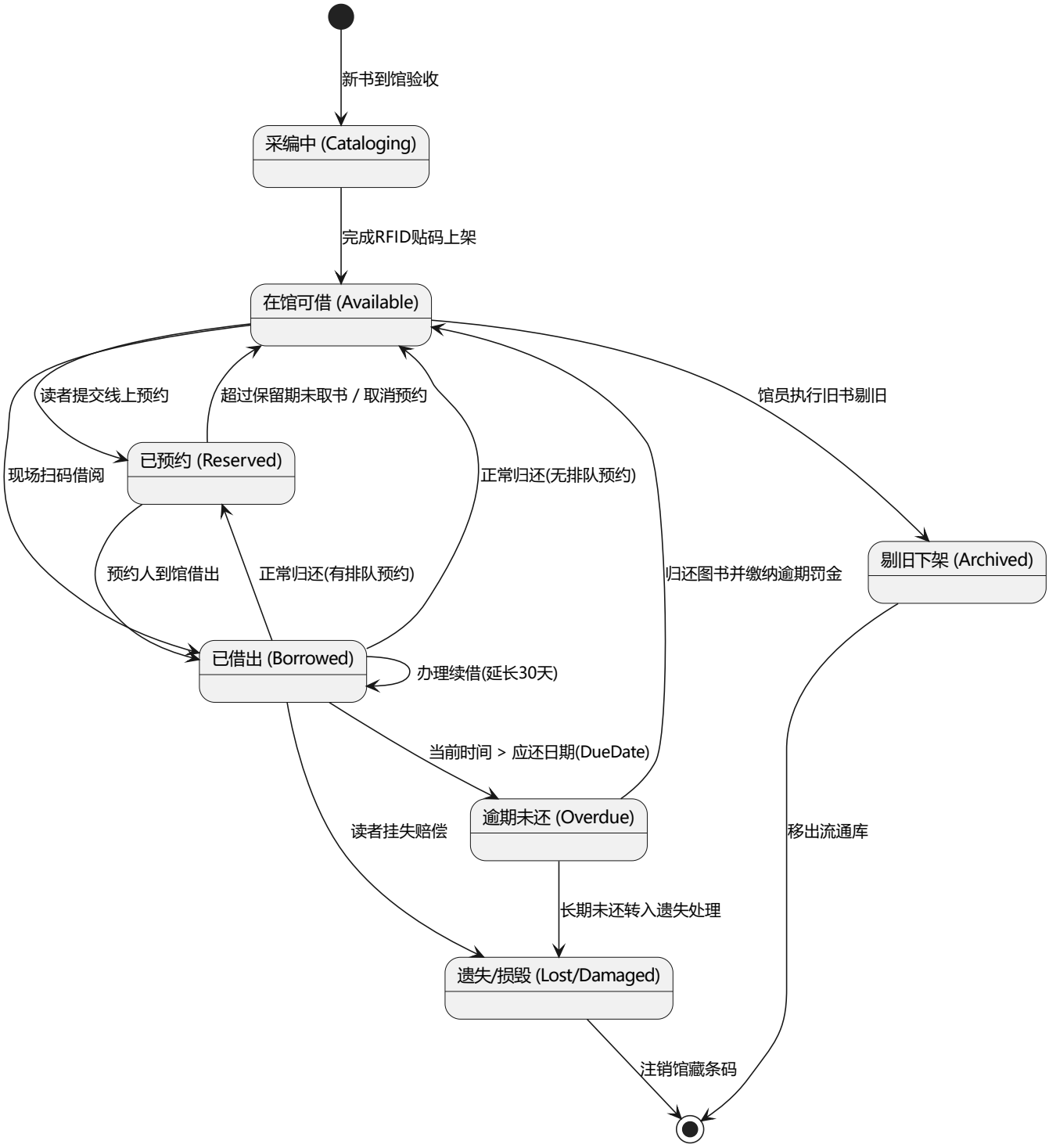
    在馆可借_Available --> 已借出_Borrowed : 读者扫描条码成功借出
    已借出_Borrowed --> 已借出_Borrowed : 读者办理续借（未逾期且无他人预约）
    已借出_Borrowed --> 在馆可借_Available : 按期归还且无人预约
    已借出_Borrowed --> 已预约锁定_Reserved : 按期归还但已有下一位读者排队预约

    已借出_Borrowed --> 逾期未还_Overdue : 超过应还日期（DueDate）未归还
    逾期未还_Overdue --> 在馆可借_Available : 归还图书并结清逾期罚款

    已借出_Borrowed --> 遗失或损毁_LostOrDamaged : 读者申报遗失 / 归还时严重破损
    逾期未还_Overdue --> 遗失或损毁_LostOrDamaged : 逾期超 90 天认定遗失
    在馆可借_Available --> 剔旧下架_Archived : 馆藏盘点老化剔旧

    遗失或损毁_LostOrDamaged --> [*] : 赔偿结案并注销条码
    剔旧下架_Archived --> [*] : 调拨基藏库或报废注销
```

PlantUML 标准版


新书到馆验收
采编中 (Cataloging)
完成RFID贴码上架
在馆可借 (Available)
读者提交线上预约
超过保留期未取书 / 取消预约
馆员执行旧书剔旧
现场扫码借阅
已预约 (Reserved)
正常归还(无排队预约)
剔旧下架 (Archived)
预约人到馆借出
正常归还(有排队预约)
归还图书并缴纳逾期罚金
已借出 (Borrowed)
办理续借(延长30天)
当前时间 > 应还日期(DueDate)
逾期未还 (Overdue)
读者挂失赔偿
移出流通库
长期未还转入遗失处理
遗失/损毁 (Lost/Damaged)
注销馆藏条码

```
@startuml
[*] --> Cataloging : 新书到馆验收

state "采编中 (Cataloging)" as Cataloging
state "在馆可借 (Available)" as Available
state "已预约 (Reserved)" as Reserved
state "已借出 (Borrowed)" as Borrowed
state "逾期未还 (Overdue)" as Overdue
state "遗失/损毁 (Lost/Damaged)" as Lost
state "剔旧下架 (Archived)" as Archived

Cataloging --> Available : 完成RFID贴码上架
Available --> Reserved : 读者提交线上预约
Reserved --> Available : 超过保留期未取书 / 取消预约
Reserved --> Borrowed : 预约人到馆借出

Available --> Borrowed : 现场扫码借阅
Borrowed --> Borrowed : 办理续借(延长30天)
Borrowed --> Available : 正常归还(无排队预约)
Borrowed --> Reserved : 正常归还(有排队预约)

Borrowed --> Overdue : 当前时间 > 应还日期(DueDate)
Overdue --> Available : 归还图书并缴纳逾期罚金

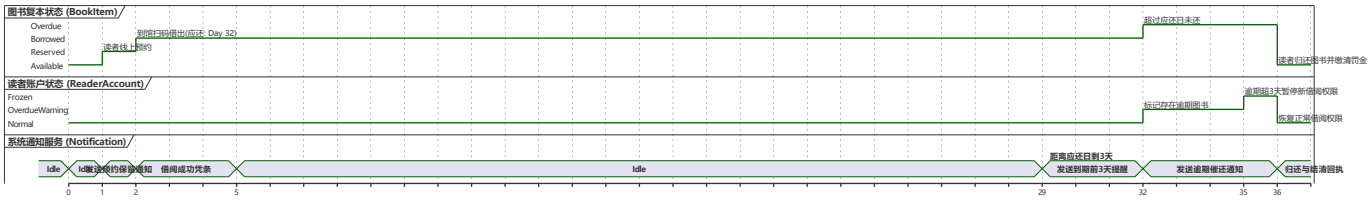
Borrowed --> Lost : 读者挂失赔偿
Overdue --> Lost : 长期未还转入遗失处理
Available --> Archived : 馆员执行旧书剔旧

Lost --> [*] : 注销馆藏条码
Archived --> [*] : 移出流通库
@enduml
```

10. Timing diagram（时间图 / 定时图）

描述在一次为期 30 天的借阅周期中，**图书复本状态（BookItem）**、**读者借阅账户状态（ReaderAccount）**以及**系统通知信号（NotificationService）**随时间轴（Day 0 预约、Day 1 借出、Day 27 临期提醒、Day 30 逾期、Day 33 归还缴罚）的精确时序状态跃迁。

PlantUML 标准版（原生 Timing Diagram 语法）




```
@startuml
robust "图书复本状态 (BookItem)" as Book
robust "读者账户状态 (ReaderAccount)" as Reader
concise "系统通知服务 (Notification)" as Notify

Book has Overdue, Borrowed, Reserved, Available
Reader has Frozen, OverdueWarning, Normal
Notify is Idle

@0
Book is Available
Reader is Normal
Notify is Idle

@1
Book is Reserved : 读者线上预约
Notify is "发送预约保留通知"

@2
Book is Borrowed : 到馆扫码借出(应还: Day 32)
Notify is "借阅成功凭条"

@5
Notify is Idle

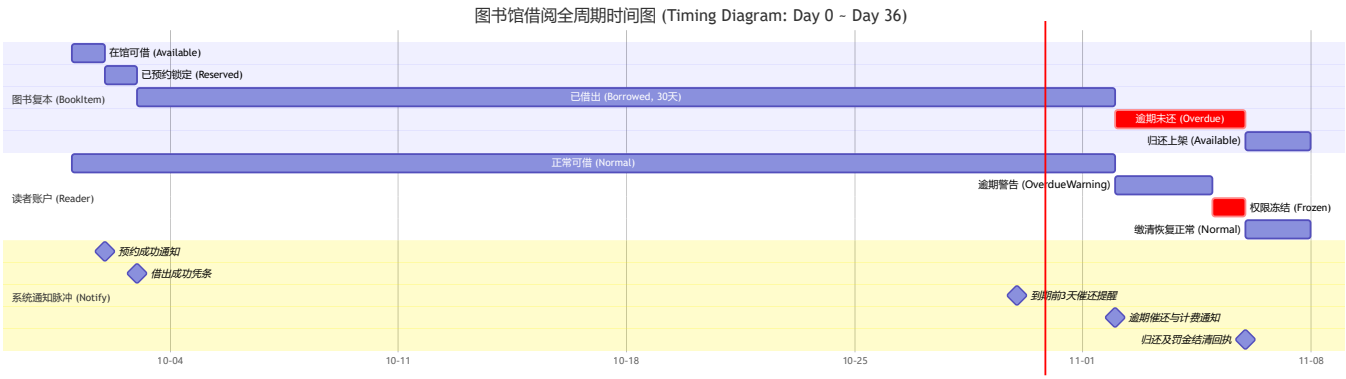
@29
Notify is "发送到期前3天提醒" : 距离应还日剩3天

@32
Book is Overdue : 超过应还日未还
Reader is OverdueWarning : 标记存在逾期图书
Notify is "发送逾期催还通知"

@35
Reader is Frozen : 逾期超3天暂停新借阅权限

@36
Book is Available : 读者归还图书并缴清罚金
Reader is Normal : 恢复正常借阅权限
Notify is "归还与结清回执"
@enduml
```

Mermaid 预览版（时间轴状态跃迁图）




```
gantt
    title 图书馆借阅全周期时间图 (Timing Diagram: Day 0 ~ Day 36)
    dateFormat  YYYY-MM-DD
    axisFormat  %m-%d

    section 图书复本 (BookItem)
    在馆可借 (Available)       :a1, 2026-10-01, 1d
    已预约锁定 (Reserved)      :a2, 2026-10-02, 1d
    已借出 (Borrowed, 30天)    :a3, 2026-10-03, 30d
    逾期未还 (Overdue)         :crit, a4, 2026-11-02, 4d
    归还上架 (Available)       :a5, 2026-11-06, 2d

    section 读者账户 (Reader)
    正常可借 (Normal)          :b1, 2026-10-01, 32d
    逾期警告 (OverdueWarning)  :b2, 2026-11-02, 3d
    权限冻结 (Frozen)          :crit, b3, 2026-11-05, 1d
    缴清恢复正常 (Normal)      :b4, 2026-11-06, 2d

    section 系统通知脉冲 (Notify)
    预约成功通知               :milestone, m1, 2026-10-02, 0d
    借出成功凭条               :milestone, m2, 2026-10-03, 0d
    到期前3天催还提醒          :milestone, m3, 2026-10-30, 0d
    逾期催还与计费通知         :milestone, m4, 2026-11-02, 0d
    归还及罚金结清回执         :milestone, m5, 2026-11-06, 0d
```